

Contents

1	Dokumentacja projektu	3
1.1	Spis treści	3
1.1.1	Dokumentacja projektowa	4
1.1.2	Dokumentacja deweloperska	4
1.1.3	Dokumentacja techniczna	4
1.1.4	Dokumentacja użytkownika	4
2	System Rezerwacji Lotów	4
2.1	2. Słownik	5
2.1.1	Użytkownik	5
2.1.2	Gość	5
2.1.3	Rezerwacja	5
2.1.4	Lot	5
2.1.5	API	5
2.1.6	Laravel	5
2.1.7	Blade	5
2.1.8	PostgreSQL	5
2.1.9	Migracja	5
2.1.10	Model	6
2.1.11	Kontroler	6
2.1.12	Widok	6
2.1.13	CRUD	6
2.1.14	Docker	6
2.1.15	Railway	6
3	1. Wstęp	6
3.1	1.1 Cel projektu	6
3.2	1.2 Przyczyna podjęcia realizacji projektu	6
3.3	1.3 Opis oprogramowania	7
3.4	1.4 Ograniczenia	7
3.5	1.5 Architektura	8
3.6	1.6 Link do działającej aplikacji	8
4	2. Wymagania funkcjonalne	8
5	3. Wymagania niefunkcjonalne	9
5.1	Interfejs	9
5.2	Dostępność i niezawodność	9
5.3	Wydażność	9
5.4	Bezpieczeństwo	9
6	4. Dokumentacja projektowa	9
6.1	4.1 Użytkownicy	9
6.1.1	Gość (Guest)	9

6.1.2	Użytkownik (User)	10
6.1.3	Administrator (opcjonalnie – jeśli masz)	10
6.2	4.2 Przypadki użycia	10
6.2.1	Aktorzy systemu	10
6.3	Diagram przypadków użycia	11
6.4	4.2.2 Tabele scenariuszy przypadków użycia	11
6.4.1	Rejestracja nowego użytkownika	11
6.4.2	Logowanie do systemu	12
6.4.3	Wyszukiwanie lotów	13
6.4.4	Rezerwacja lotu	13
6.4.5	Zarządzanie kontem	14
6.5	4.2 Diagramy czynności	15
6.5.1	Wyszukiwanie	15
6.5.2	Rezerwacja	16
6.6	4.3 Modele bazy danych	17
6.6.1	Model koncepcyjny bazy danych	17
6.6.2	Encje:	17
6.6.3	Relacje:	17
6.7	Model logiczny bazy danych	18
6.8	Model fizyczny bazy danych	19
6.9	4.4 Diagramy stanów	19
6.9.1	Logowanie	20
6.9.2	Rejestracja	21
6.9.3	Rezerwacja	21
6.10	4.5 Diagramy sekwencji	21
6.10.1	Rezerwacja lotu	22
6.10.2	wyszukiwanie lotów	22
6.11	5. Dokumentacja bezpieczeństwa systemu	22
6.11.1	5.1 Bezpieczeństwo danych w bazie	22
6.11.2	5.2 Bezpieczeństwo transmisji danych	22
6.11.3	5.3 Secure by Design	23
6.11.4	5.4 Privacy by Design	23
6.11.5	5.5 Zero Trust	23
6.12	6. Rozwiązania zwiększające dostępność (WCAG)	23
7	Dokumentacja deweloperska	24
7.1	7. Diagram klas	24
7.2	8. Kod SQL	24
7.2.1	Standard SQL do tworzenia modelu bazy danych	24
7.2.2	Dialekt SQL	25
7.3	Przypadki testowe	25
7.3.1	Przypadek testowy 1 – Logowanie użytkownika	25
7.3.2	Przypadek testowy 2 – Utworzenie rezerwacji lotu	26
7.4	Testy jednostkowe	26
7.4.1	Test jednostkowy 1 – Tworzenie użytkownika	26
7.4.2	Wynik testu	27

7.4.3	Test jednostkowy 2 – Status nowej rezerwacji	27
7.4.4	Wynik testu	28
7.5	Testy integracyjne	28
7.5.1	Zakres testów integracyjnych	29
7.5.2	Test 1 – Tworzenie rezerwacji	29
7.5.3	Test 2 – Relacja użytkownik → rezerwacje	30
7.5.4	Wynik testów integracyjnych	30
7.6	Instalacja i konfiguracja:	30
7.6.1	Continuous Integration i Continuous Deployment (CI/CD)	30
7.7	Implementacja zaplanowanych mechanizmów zapewniających bezpieczeństwo w praktyce	31
7.7.1	1. Bezpieczeństwo danych w tranzycie i spoczynku	31
7.7.2	2. Autoryzacja i uwierzytelnianie	31
7.7.3	3. Ochrona przed atakami	31
7.7.4	4. Bezpieczeństwo infrastruktury	31
7.7.5	5. CI/CD i bezpieczeństwo wdrożeń	31
7.7.6	6. Ochrona danych użytkownika (Privacy by Design)	32
8	Dokumentacja użytkownika	32
8.1	Spis treści	32
8.2	1. Wprowadzenie	32
8.3	2. Rejestracja i logowanie	32
8.4	3. Wyszukiwanie lotów	33
9	4. Proces rezerwacji lotu	33
9.0.1	Krok 1 – wybór lotu	33
9.0.2	Krok 2 – dane pasażerów	33
9.0.3	Krok 3 – dane kontaktowe	33
9.0.4	Krok 4 – podsumowanie	33
9.0.5	Krok 5 – potwierdzenie	33
10	5. Zarządzanie rezerwacjami	33
10.0.1	Dostępne operacje:	33
10.0.2	Statusy rezerwacji:	34
10.0.3	Anulowanie rezerwacji	34
10.1	6. Płatności	34
10.2	7. Generowanie biletu PDF	34
10.3	8. Panel użytkownika	34
10.4	9. Rozwiązywanie problemów	34

1 Dokumentacja projektu

1.1 Spis treści

1. Strona tytułowa
2. Słownik

3. Cel i zakres projektu
4. Architektura systemu
5. Wymagania
 - 5.1 Wymagania funkcjonalne
 - 5.2 Wymagania niefunkcjonalne
6. Ograniczenia

1.1.1 Dokumentacja projektowa

7. Użytkownicy
8. Przypadki użycia
9. Baza danych
10. Diagramy sekwencji
11. Diagramy aktywności
12. Diagramy stanów
13. Dokumentacja bezpieczeństwa
14. Dostępność (WCAG)

1.1.2 Dokumentacja deweloperska

15. Diagram klas
16. Kod SQL
17. Przypadki testowe
18. Testy jednostkowe

1.1.3 Dokumentacja techniczna

19. Diagram komponentów i wdrożenia
20. Instalacja i konfiguracja
21. Implementacja mechanizmów bezpieczeństwa

1.1.4 Dokumentacja użytkownika

22. Podręcznik użytkownika

2 System Rezerwacji Lotów

Nazwa projektu: System Rezerwacji Lotów

Skrót projektu: FlightBookingSystem

Autorzy: - Patryk - Denis Stefański

Repozytorium projektu: <https://github.com/DzikiP/System-rezerwacji-lotow>

Technologie: - Laravel 12 - PHP 8.2 - PostgreSQL - Blade - Tailwind CSS - Docker

2.1 2. Słownik

Poniżej przedstawiono podstawowe pojęcia używane w dokumentacji oraz w systemie.

2.1.1 Użytkownik

Osoba korzystająca z systemu. Może przeglądać loty, rezerwować je oraz zarządzać swoimi rezerwacjami po zalogowaniu.

2.1.2 Gość

Niezalogowany użytkownik systemu, który ma dostęp jedynie do przeglądania i wyszukiwania lotów.

2.1.3 Rezerwacja

Zapis w systemie reprezentujący wybór konkretnego lotu przez użytkownika wraz z jego danymi.

2.1.4 Lot

Pojedyncze połączenie lotnicze pobierane z zewnętrznego API, zawierające informacje takie jak trasa, godziny oraz cena.

2.1.5 API

Interfejs programistyczny umożliwiający komunikację z zewnętrznym systemem dostarczającym dane o lotach.

2.1.6 Laravel

Framework backendowy w języku PHP wykorzystywany do implementacji logiki aplikacji oraz obsługi żądań HTTP.

2.1.7 Blade

Silnik szablonów używany w Laravelu do generowania dynamicznych widoków HTML.

2.1.8 PostgreSQL

Relacyjny system zarządzania bazą danych wykorzystywany do przechowywania danych aplikacji, takich jak użytkownicy i rezerwacje.

2.1.9 Migracja

Mechanizm w Laravelu pozwalający na wersjonowanie i automatyczne tworzenie struktury bazy danych.

2.1.10 Model

Klasa w architekturze MVC reprezentująca dane oraz logikę operacji na nich (np. użytkownik, rezerwacja).

2.1.11 Kontroler

Warstwa aplikacji odpowiedzialna za obsługę żądań użytkownika oraz komunikację między widokiem a modelem.

2.1.12 Widok

Warstwa prezentacji w architekturze MVC odpowiedzialna za interfejs użytkownika.

2.1.13 CRUD

Zestaw operacji: Create, Read, Update, Delete, czyli podstawowe operacje wykonywane na danych w systemie.

2.1.14 Docker

Narzędzie do konteneryzacji aplikacji, umożliwiające jej uruchamianie w izolowanym środowisku.

2.1.15 Railway

Platforma chmurowa wykorzystywana do wdrożenia (deploymentu) aplikacji.

3 1. Wstęp

3.1 1.1 Cel projektu

Celem projektu jest zaprojektowanie i implementacja aplikacji internetowej umożliwiającej wyszukiwanie połączeń lotniczych oraz zarządzanie rezerwacjami biletów lotniczych.

System umożliwia użytkownikom przeglądanie dostępnych lotów, ich wyszukiwanie na podstawie określonych parametrów oraz tworzenie i zarządzanie rezerwacjami.

3.2 1.2 Przyczyna podjęcia realizacji projektu

Główną przyczyną realizacji projektu jest potrzeba informatyzacji procesu wyszukiwania i rezerwacji lotów.

Współczesne systemy rezerwacyjne wymagają szybkiego dostępu do danych oraz intuicyjnego interfejsu użytkownika. Zastosowanie aplikacji webowej umożliwia:

- uproszczenie procesu wyszukiwania lotów,
 - centralizację danych o rezerwacjach użytkowników,
 - zwiększenie wygody użytkowników poprzez dostęp do systemu z poziomu przeglądarki internetowej,
 - automatyzację procesu zarządzania rezerwacjami.
-

3.3 1.3 Opis oprogramowania

Oprogramowanie realizuje proces wyszukiwania oraz rezerwacji lotów.

Użytkownik może wyszukiwać loty na podstawie miejsca wylotu, miejsca docelowego oraz daty. System integruje się z zewnętrznym API dostarczającym dane o dostępnych połączeniach lotniczych.

Dodatkowo aplikacja umożliwia: - tworzenie kont użytkowników, - logowanie i autoryzację, - tworzenie rezerwacji lotów, - przegląd i zarządzanie rezerwacjami, - edycję oraz usuwanie rezerwacji.

System przechowuje dane użytkowników oraz rezerwacji w relacyjnej bazie danych PostgreSQL.

3.4 1.4 Ograniczenia

- Aplikacja działa wyłącznie w przeglądarce internetowej
 - System wymaga aktywnego połączenia z Internetem
 - Dane o lotach zależą od dostępności zewnętrznego API
 - Brak integracji z systemami płatności online
 - System nie gwarantuje rzeczywistej dostępności miejsc w czasie rzeczywistym
 - Aplikacja została zaprojektowana w architekturze MVC (Laravel)
 - System działa w środowisku PHP 8.2
 - Wymagana baza danych: PostgreSQL
-

3.5 1.5 Architektura

Aplikacja została zbudowana w oparciu o architekturę MVC (Model–View–Controller).

System składa się z następujących warstw:

- **Warstwa prezentacji (View)** – interfejs użytkownika oparty o Blade i Tailwind CSS
 - **Warstwa kontrolerów (Controller)** – obsługa żądań HTTP oraz logika przepływu aplikacji
 - **Warstwa modelu (Model)** – reprezentacja danych oraz komunikacja z bazą danych
 - **Warstwa zewnętrznych usług (API)** – integracja z zewnętrznym API lotów
 - **Warstwa bazy danych** – PostgreSQL przechowujący dane użytkowników i rezerwacji
-

3.6 1.6 Link do działającej aplikacji

Aplikacja działająca: *system-rezerwacji-lotow-production.up.railway.app*

:contentReferenceoaicite:0

4 2. Wymagania funkcjonalne

- System umożliwia wyszukiwanie dostępnych lotów na podstawie miejsca wylotu, miejsca docelowego oraz daty.
 - System umożliwia przeglądanie wyników wyszukiwania lotów przez użytkowników niezalogowanych.
 - System umożliwia rejestrację użytkownika.
 - System umożliwia logowanie użytkownika do systemu.
 - System umożliwia tworzenie rezerwacji wybranego lotu przez zalogowanego użytkownika.
 - System umożliwia przegląd listy rezerwacji użytkownika.
 - System umożliwia edycję oraz usuwanie rezerwacji.
 - System przechowuje historię rezerwacji użytkownika.
 - System umożliwia wylogowanie użytkownika.
 - System integruje się z zewnętrznym API dostarczającym dane o lotach.
-

5 3. Wymagania niefunkcjonalne

5.1 Interfejs

- Aplikacja działa jako strona internetowa (web application).
 - Interfejs użytkownika jest responsywny (RWD).
 - Interfejs został wykonany z użyciem Tailwind CSS.
 - System zapewnia spójny wygląd wszystkich widoków.
-

5.2 Dostępność i niezawodność

- System działa w trybie 24/7.
 - System wymaga połączenia z Internetem.
 - System jest kompatybilny z popularnymi przeglądarkami (Chrome, Edge, Firefox).
 - Aplikacja działa poprawnie na urządzeniach mobilnych i desktopowych.
-

5.3 Wydajność

- Czas odpowiedzi systemu na podstawowe zapytania nie przekracza 1–2 sekund.
 - System jest w stanie obsłużyć wielu równoczesnych użytkowników (ograniczone zasobami hostingu).
-

5.4 Bezpieczeństwo

- Hasła użytkowników są przechowywane w postaci zaszyfrowanej (hash).
 - Dostęp do funkcji systemu wymaga autoryzacji.
 - System chroni dane użytkowników przed nieautoryzowanym dostępem.
 - Komunikacja odbywa się przez bezpieczny protokół HTTPS (w środowisku produkcyjnym).
 - Dane sesyjne są chronione przed nieautoryzowanym dostępem.
-

6 4. Dokumentacja projektowa

6.1 4.1 Użytkownicy

6.1.1 Gość (Guest)

Niealogowany użytkownik systemu. Może przeglądać oraz wyszukiwać loty, ale nie ma możliwości dokonywania rezerwacji.

6.1.2 Użytkownik (User)

Zalogowany użytkownik systemu. Może wyszukiwać loty, tworzyć rezerwacje oraz zarządzać swoimi rezerwacjami.

6.1.3 Administrator (opcjonalnie – jeśli masz)

Użytkownik posiadający pełne uprawnienia do zarządzania systemem, użytkownikami oraz rezerwacjami.

6.2 4.2 Przypadki użycia

System umożliwia realizację procesów związanych z wyszukiwaniem lotów oraz zarządzaniem rezerwacjami.

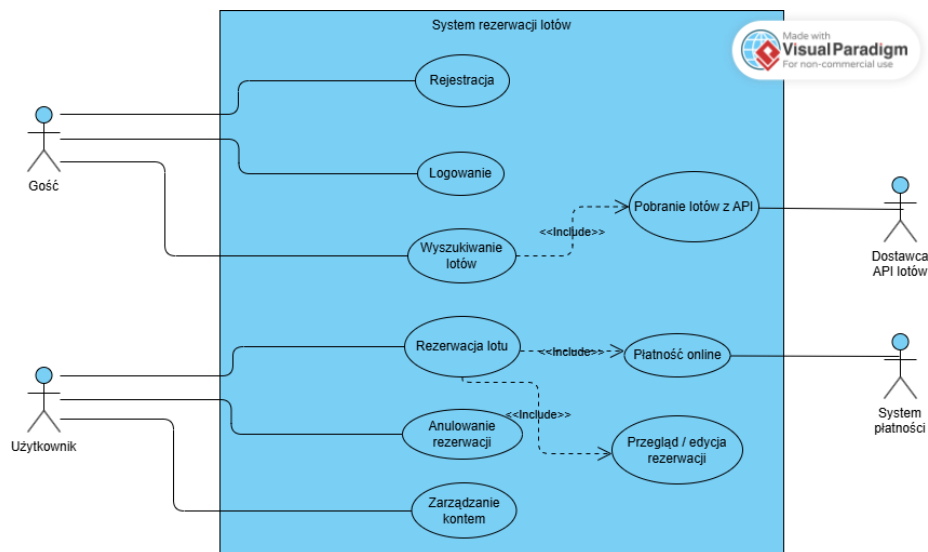
6.2.1 Aktorzy systemu

- **Gość (Guest)** – użytkownik niezalogowany
- **Użytkownik (User)** – użytkownik zalogowany
- **System zewnętrzny (API lotów)** – dostarcza dane o lotach

W systemie zidentyfikowano następujące przypadki użycia:

ID	Przypadek użycia	Aktor
UC-01	Rejestracja użytkownika	Gość
UC-02	Logowanie do systemu	Użytkownik
UC-03	Wylogowanie z systemu	Użytkownik
UC-04	Wyszukiwanie lotów	Gość, Użytkownik
UC-05	Wyświetlenie szczegółów lotu	Gość, Użytkownik
UC-06	Utworzenie rezerwacji lotu	Użytkownik
UC-07	Wyświetlenie listy rezerwacji	Użytkownik
UC-08	Edycja rezerwacji	Użytkownik
UC-09	Usunięcie (anulowanie) rezerwacji	Użytkownik
UC-10	Zarządzanie kontem użytkownika	Użytkownik

6.3 Diagram przypadków użycia



Rys 1. Diagram przypadków użycia systemu rezerwacji lotów

6.4 4.2.2 Tabele scenariuszy przypadków użycia

6.4.1 Rejestracja nowego użytkownika

Element	Opis
Cel w kontekście systemu	Umożliwienie użytkownikowi utworzenia konta w systemie rezerwacji lotów
Warunki wstępne	Użytkownik posiada dostęp do Internetu oraz aktywny adres e-mail
Warunek pomyślnego zakończenia	Konto użytkownika zostaje utworzone i aktywowane
Stan końcowy – niepowodzenie	Rejestracja nie zostaje ukończona z powodu błędnych danych lub braku aktywacji konta
Główni aktorzy	Użytkownik
Aktorzy współuczestniczący	System rezerwacji lotów, serwer poczty e-mail

Element	Opis
Wywołanie (inicjacja)	Użytkownik wybiera opcję „Zarejestruj się”
Include	Walidacja danych, wysyłka e-maila aktywacyjnego
Extend	Logowanie po aktywacji konta
Scenariusz główny	1. Użytkownik otwiera formularz rejestracji 2. Wprowadza dane (email, hasło, imię, nazwisko) 3. System waliduje dane 4. System zapisuje użytkownika 5. System wysyła e-mail aktywacyjny 6. Użytkownik aktywuje konto
Scenariusze alternatywne	A1: Błędne dane – system wyświetla komunikat błędu A2: Brak aktywacji – konto nie zostaje aktywowane

6.4.2 Logowanie do systemu

Element	Opis
Cel w kontekście systemu	Umożliwienie użytkownikowi zalogowania się do systemu
Warunki wstępne	Konto użytkownika istnieje i jest aktywne
Warunek pomyślnego zakończenia	Użytkownik uzyskuje dostęp do konta
Stan końcowy – niepowodzenie	Logowanie nie powiodło się (błędne dane lub nieaktywne konto)
Główni aktorzy	Użytkownik
Aktorzy współuczestniczący	System rezerwacji lotów, baza danych
Wywołanie (inicjacja)	Użytkownik wybiera „Zaloguj się”
Include	Weryfikacja danych logowania
Extend	Reset hasła
Scenariusz główny	1. Użytkownik wpisuje login i hasło 2. System weryfikuje dane 3. System tworzy sesję 4. Użytkownik zostaje zalogowany

Element	Opis
Scenariusze alternatywne	A1: Błędne dane – komunikat o błędzie A2: Konto nieaktywne – brak dostępu

6.4.3 Wyszukiwanie lotów

Element	Opis
Cel w kontekście systemu	Umożliwienie wyszukiwania dostępnych lotów
Warunki wstępne	Użytkownik ma dostęp do wyszukiwarki
Warunek pomyślnego zakończenia	System wyświetla listę dostępnych lotów
Stan końcowy – niepowodzenie	Brak wyników lub błąd API
Główni aktorzy	Użytkownik
Aktorzy współuczestniczący	Zewnętrzne API lotów
Wywołanie (inicjacja)	Użytkownik wprowadza parametry wyszukiwania
Include	Pobieranie danych z API, filtrowanie wyników
Extend	Rezerwacja lotu, szczegóły lotu
Scenariusz główny	1. Użytkownik wpisuje kryteria wyszukiwania 2. System wysyła zapytanie do API 3. API zwraca dane 4. System wyświetla wyniki
Scenariusze alternatywne	A1: Brak wyników – komunikat A2: Błąd API – ponowienie zapytania

6.4.4 Rezerwacja lotu

Element	Opis
Cel w kontekście systemu	Utworzenie rezerwacji wybranego lotu
Warunki wstępne	Użytkownik wybrał lot i jest zalogowany

Element	Opis
Warunek pomyślnego zakończenia	Rezerwacja zostaje zapisana w systemie
Stan końcowy – niepowodzenie	Rezerwacja anulowana lub nieukończona
Główni aktorzy	Użytkownik
Aktorzy współuczestniczący	System rezerwacji lotów
Wywołanie (inicjacja)	Użytkownik wybiera „Zarezerwuj”
Include	Walidacja danych, zapis rezerwacji
Extend	Anulowanie rezerwacji
Scenariusz główny	1. Użytkownik wybiera lot 2. System pokazuje szczegóły 3. Użytkownik potwierdza 4. System zapisuje rezerwację
Scenariusze alternatywne	A1: Brak logowania – przekierowanie do logowania

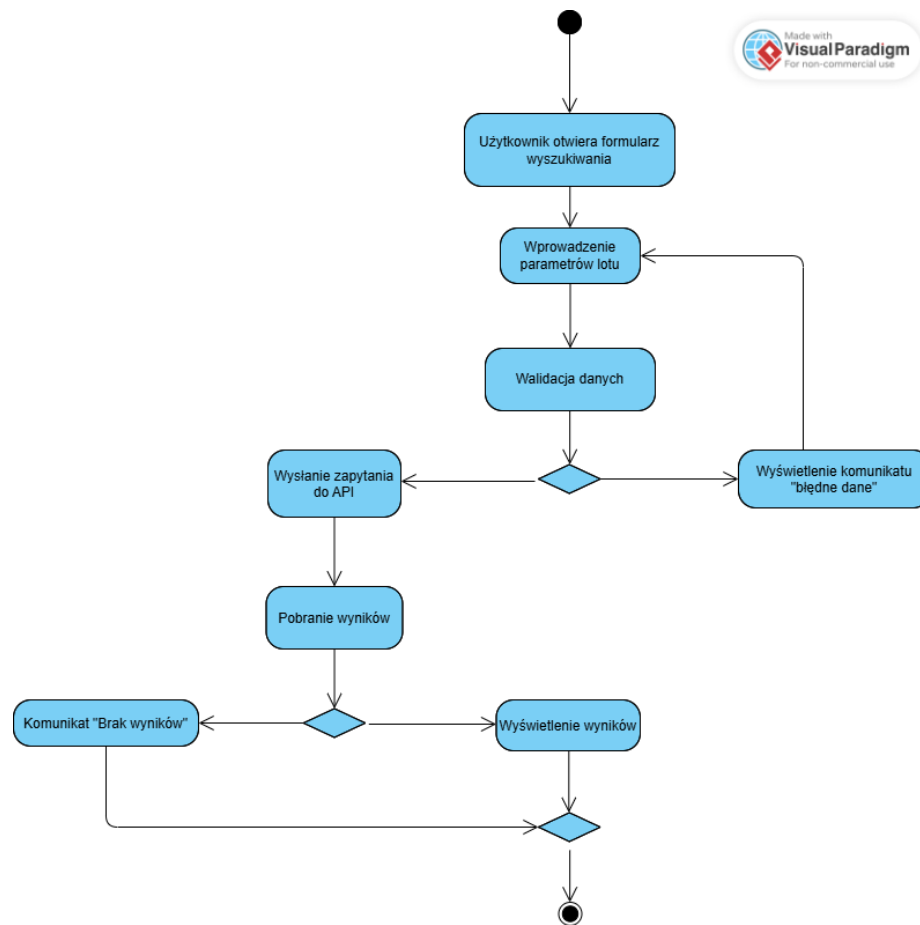
6.4.5 Zarządzanie kontem

Element	Opis
Cel w kontekście systemu	Zarządzanie danymi użytkownika i jego rezerwacjami
Warunki wstępne	Użytkownik jest zalogowany
Warunek pomyślnego zakończenia	Dane zostają zaktualizowane
Stan końcowy – niepowodzenie	Brak uprawnień lub błąd systemu
Główni aktorzy	Użytkownik
Aktorzy współuczestniczący	System rezerwacji lotów
Wywołanie (inicjacja)	Użytkownik otwiera panel konta
Include	Edycja danych, przegląd rezerwacji
Extend	Anulowanie rezerwacji
Scenariusz główny	1. Użytkownik otwiera konto 2. System wyświetla dane 3. Użytkownik edytuje dane lub rezerwacje 4. System zapisuje zmiany
Scenariusze alternatywne	A1: Brak uprawnień – komunikat A2: Błąd zapisu – ponowienie operacji

6.5 4.2 Diagramy czynności

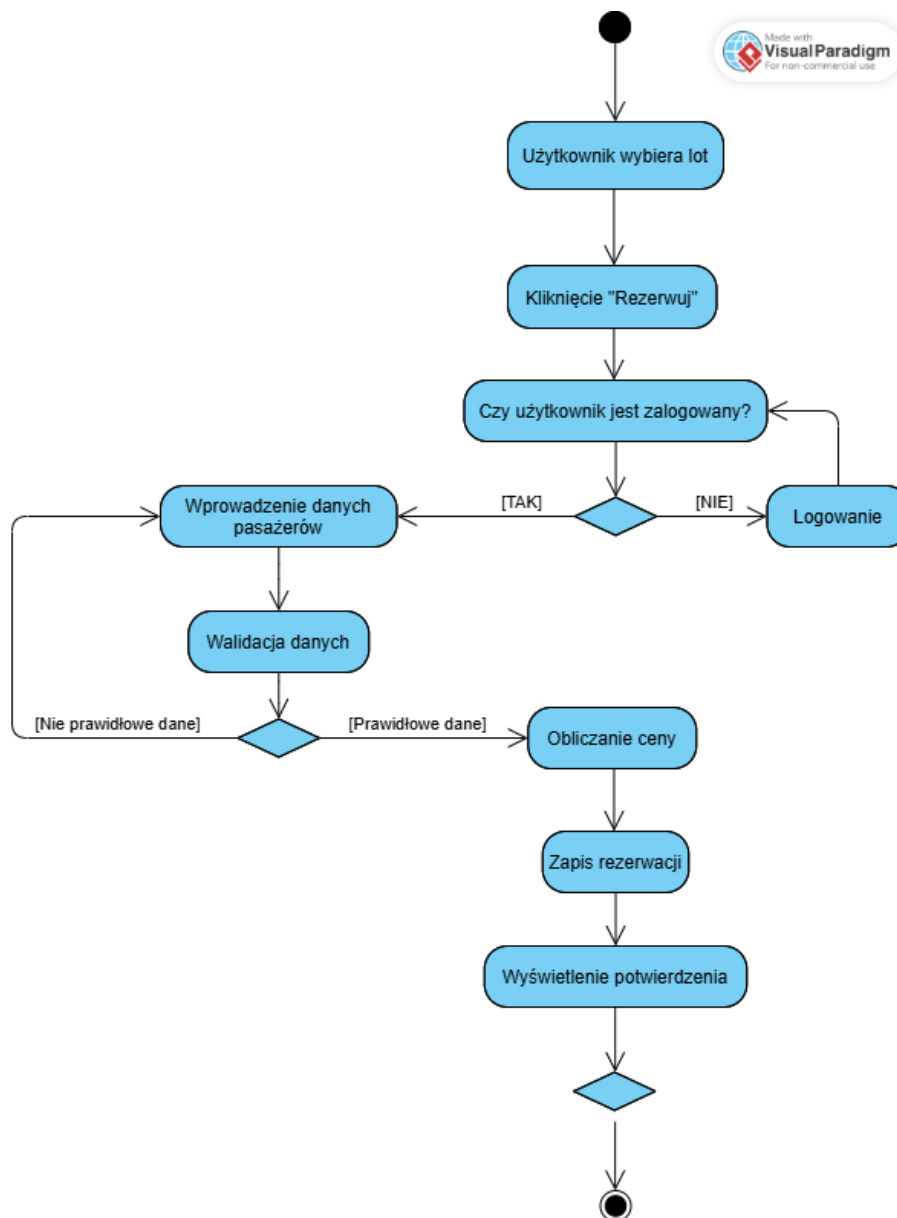
- Rejestracja użytkownika
- Logowanie do systemu
- Wyszukiwanie lotów
- Rezerwacja lotu
- Edycja rezerwacji
- Anulowanie rezerwacji
- Zarządzanie kontem użytkownika

6.5.1 Wyszukiwanie



Rys. 2 diagram czynności wyszukiwania

6.5.2 Rezerwacja



Rys. 3 diagram czynności rezerwacja

6.6 4.3 Modele bazy danych

6.6.1 Model koncepcyjny bazy danych

6.6.2 Encje:

- Użytkownik (User)
- Rola (Role)
- Rezerwacja (Booking)
- Pasażer (Passenger)
- Bilet (Ticket)
- Płatność (Payment)
- Lotnisko (Airport)

6.6.3 Relacje:

- Rola przypisana jest do wielu użytkowników (1:N)
- Użytkownik może posiadać wiele rezerwacji (1:N)
- Rezerwacja zawiera wielu pasażerów (1:N)
- Rezerwacja może mieć jedną płatność (1:0..1)
- Rezerwacja może generować jeden bilet (1:0..1)
- Lotnisko funkcjonuje jako encja referencyjna (bez relacji w aktualnej wersji systemu)

Model koncepcyjny odzwierciedla strukturę systemu z punktu widzenia biznesowego.

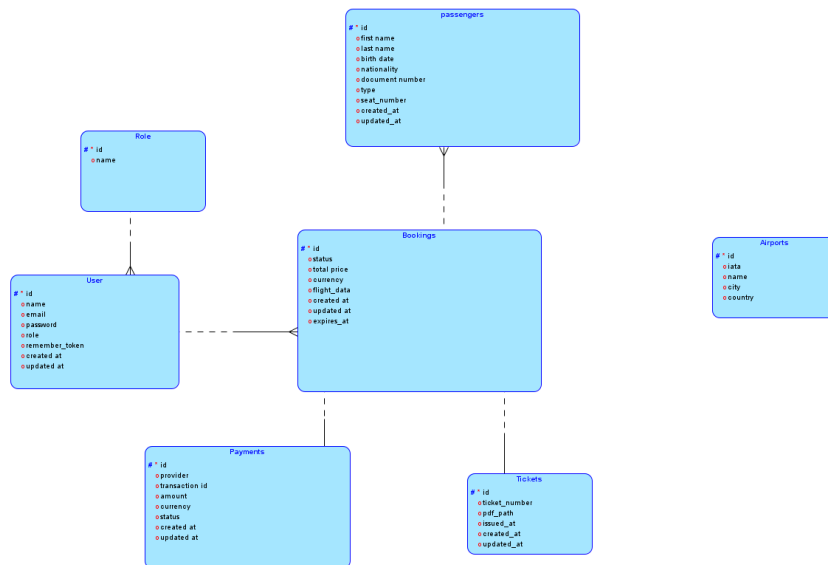
Model bazy danych został zaprojektowany z wyłączeniem kolumny `flight_data`, która została celowo zdenormalizowana w celu przechowywania danych pochodzących z zewnętrznego API w formacie JSON.

Tabela `airports` pełni rolę słownika referencyjnego zawierającego dane o lotniskach (IATA, ICAO, nazwa, lokalizacja).

W aktualnej architekturze systemu tabela nie posiada relacji z innymi encjami, ponieważ dane o lotach są pobierane z zewnętrznego API i przechowywane w formie JSON w kolumnie `flight_data` tabeli `bookings`.

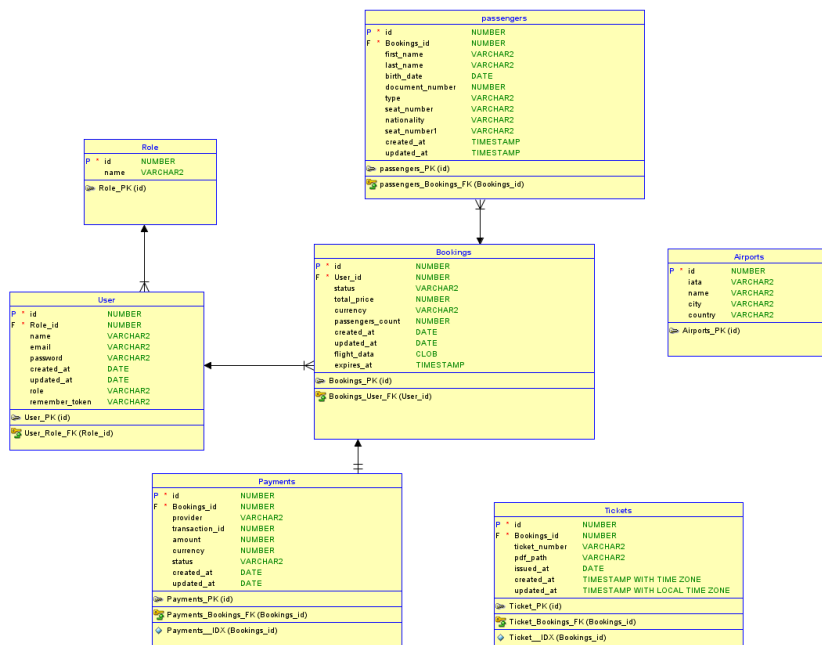
Tabela może być wykorzystywana jako cache lub źródło danych pomocniczych w procesie wyszukiwania lotów.

6.7 Model logiczny bazy danych



Rys. 4 Model logiczny bazy danych systemu rezerwacji lotów

6.8 Model fizyczny bazy danych

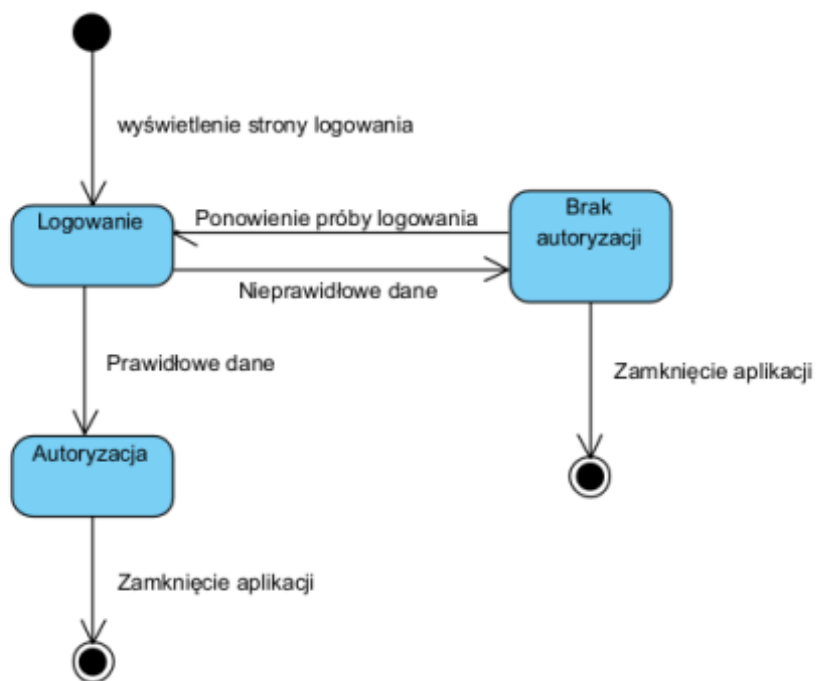


Rys. 5 Model logiczny bazy danych systemu rezerwacji lotów

6.9 4.4 Diagramy stanów

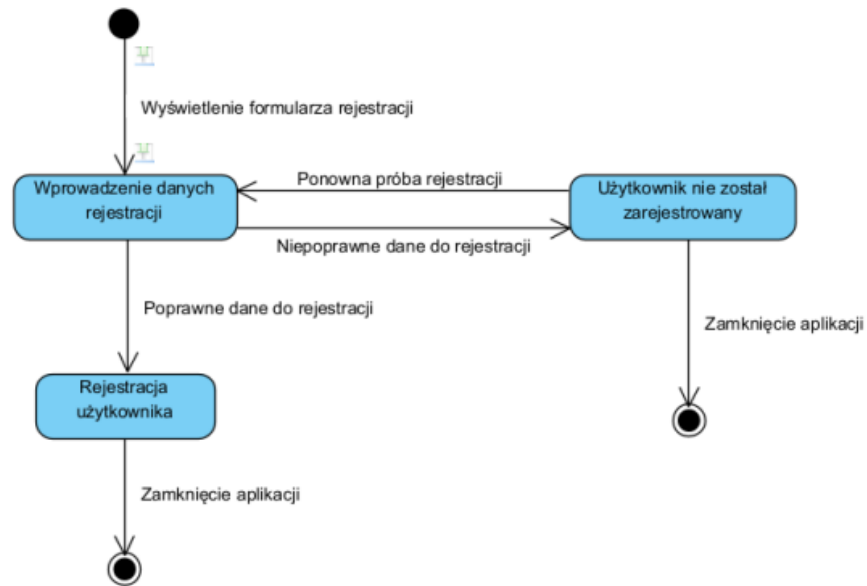
- Stan rezerwacji
- Stan płatności
- Stan biletu
- konta użytkownika

6.9.1 Logowanie



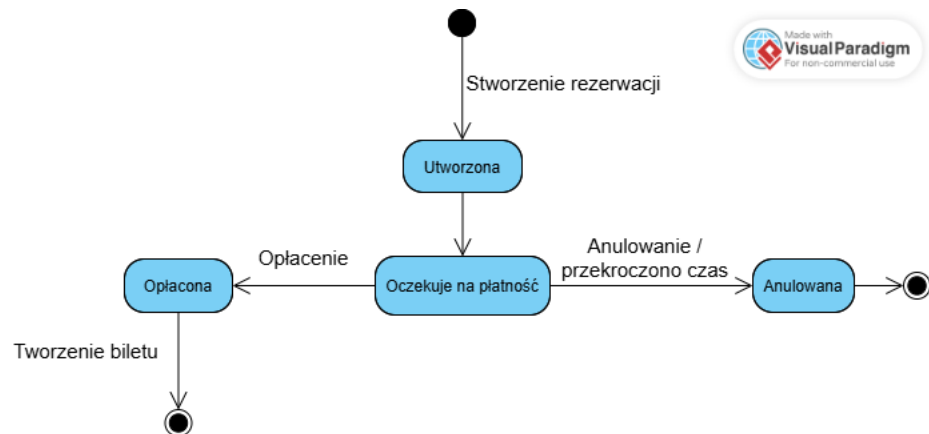
Rys. 6 Diagram stanu logowania

6.9.2 Rejestracja



Rys. 7 Diagram stanu rejestracji

6.9.3 Rezerwacja



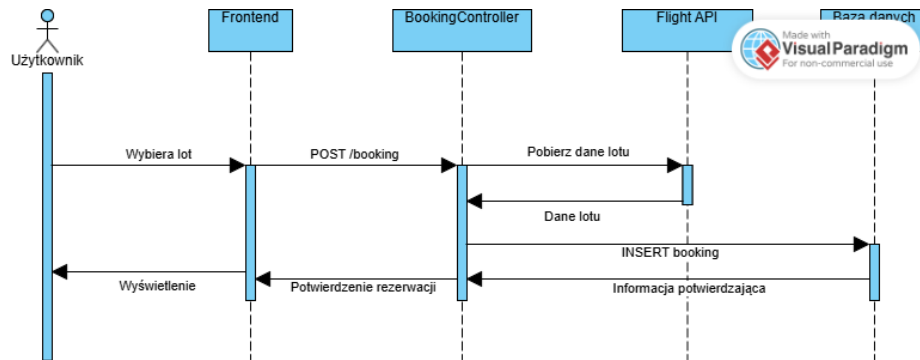
Rys. 8 Diagram stanu rezerwacji

6.10 4.5 Diagramy sekwencji

- Rejestracja użytkownika
- Logowanie do systemu
- Wyszukiwanie lotów

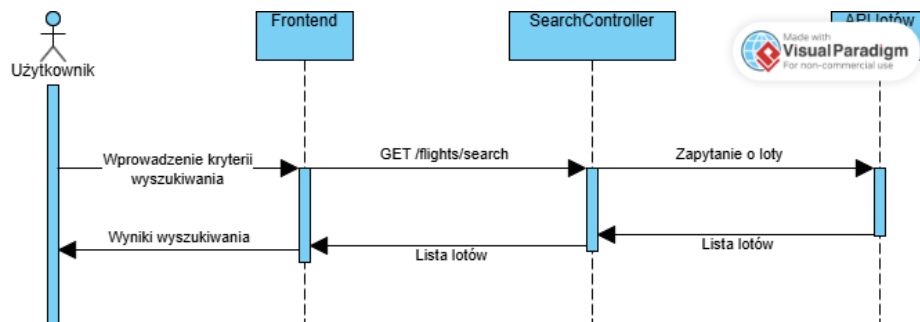
- Rezerwacja lotu
- Edycja rezerwacji
- Anulowanie rezerwacji
- Generowanie biletu

6.10.1 Rezerwacja lotu



Rys. 9 Diagram sekwencji rejestracji

6.10.2 wyszukiwanie lotów



Rys. 10 Diagram sekwencji wyszukiwania

6.11 5. Dokumentacja bezpieczeństwa systemu

6.11.1 5.1 Bezpieczeństwo danych w bazie

- Hasła użytkowników są przechowywane jako hash (bcrypt).
- Dane w bazie są chronione przez kontrolę dostępu i uprawnienia (RBAC).
- Dostęp do danych ograniczony jest zasadą least privilege.
- Dane rezerwacji i pasażerów są powiązane kluczami obcymi, co ogranicza dostęp nieautoryzowany.

6.11.2 5.2 Bezpieczeństwo transmisji danych

- Komunikacja odbywa się przez HTTPS (TLS).

- Dane logowania przesyłane są metodą POST.
- System nie przekazuje danych w URL.
- Integracje z API zewnętrznymi również korzystają z HTTPS.

6.11.3 5.3 Secure by Design

- walidacja danych wejściowych,
- ochrona przed SQL Injection (Eloquent ORM),
- ochrona przed XSS),
- kontrola dostępu oparta o role (admin/user),
- separacja logiki w architekturze MVC.

6.11.4 5.4 Privacy by Design

- zbierane są tylko niezbędne dane użytkowników,
- dane pasażerów ograniczone do minimum,
- brak udostępniania danych podmiotom trzecim poza API,
- użytkownik ma kontrolę nad swoimi danymi.

6.11.5 5.5 Zero Trust

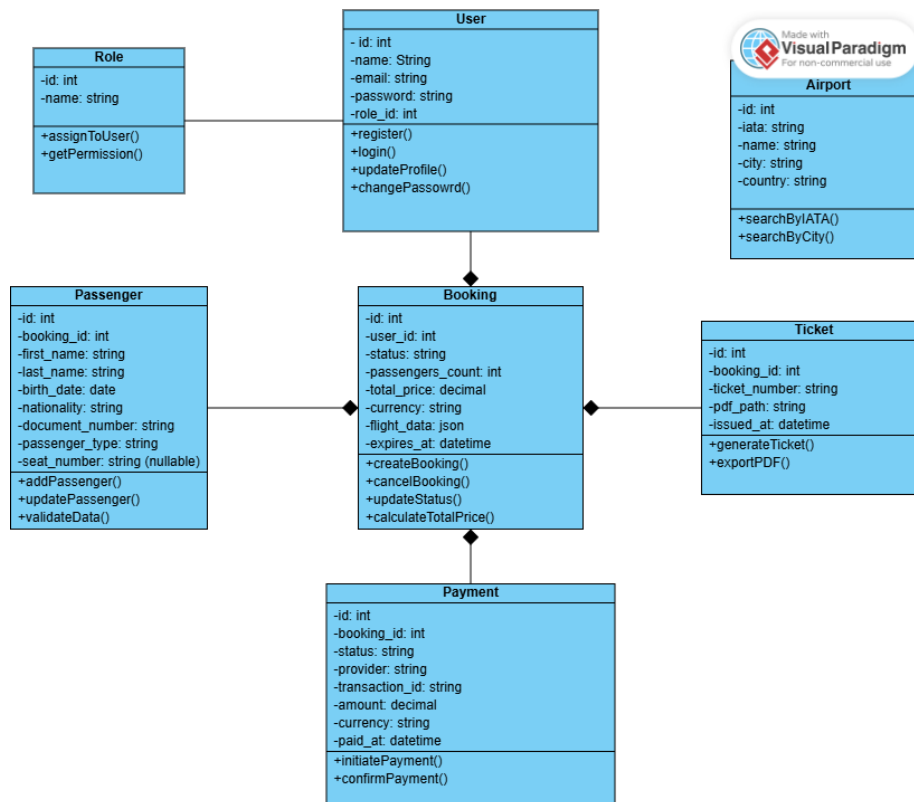
- każde żądanie jest weryfikowane niezależnie,
- system nie ufa danym z frontendu,
- dostęp zależy od ról i uprawnień,

6.12 6. Rozwiązania zwiększające dostępność (WCAG)

System w podstawowym zakresie spełnia wymagania WCAG 2.1 na poziomie A, z częściowym uwzględnieniem poziomu AA, w szczególności w zakresie czytelności, nawigacji oraz obsługi klawiaturą.

7 Dokumentacja deweloperska

7.1 7. Diagram klas



Rys. 11 Diagram klas

7.2 8. Kod SQL

7.2.1 Standard SQL do tworzenia modelu bazy danych

Model bazy danych został zaimplementowany przy użyciu migracji frameworka Laravel. Migracje definiują strukturę tabel, klucze główne, klucze obce oraz ograniczenia integralności danych.

Pełny kod SQL modelu bazy danych znajduje się w katalogu `database/migrations` repozytorium projektu.

Repozytorium projektu: <https://github.com/DzikiP/System-rezerwacji-lotow>

Poniżej przedstawiono przykładową definicję tabeli `users`.

```
CREATE TABLE roles (
    id BIGSERIAL PRIMARY KEY,
```



```

        name VARCHAR(255) UNIQUE NOT NULL
    );

CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    role_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20),

    CONSTRAINT fk_users_role
        FOREIGN KEY (role_id)
        REFERENCES roles(id)
);

```

7.2.2 Dialekt SQL

Projekt wykorzystuje bazę danych PostgreSQL, dlatego zastosowany został dialekt PostgreSQL SQL.

W warstwie aplikacji dostęp do bazy danych realizowany jest za pomocą ORM Eloquent dostarczanego przez framework Laravel. Struktura bazy definiowana jest przy użyciu migracji Laravel, które podczas wykonywania generują odpowiedni kod SQL zgodny z dialektem PostgreSQL.

7.3 Przypadki testowe

7.3.1 Przypadek testowy 1 – Logowanie użytkownika

Pole	Opis
Nazwa	Logowanie użytkownika
Cel	Sprawdzenie poprawności logowania użytkownika do systemu.
Warunki wstępne	Użytkownik posiada aktywne konto w systemie.
Dane testowe	E-mail: <code>test@test.pl</code> Hasło: <code>password123</code>
Kroki	1. Otwórz stronę logowania. 2. Wprowadź poprawny adres e-mail i hasło. 3. Kliknij przycisk Zaloguj .
Oczekiwany rezultat	Użytkownik zostaje zalogowany i przekierowany do panelu głównego.
Wynik testu	Pozytywny

7.3.2 Przypadek testowy 2 – Utworzenie rezerwacji lotu

Pole	Opis
Nazwa	Utworzenie rezerwacji lotu
Cel	Sprawdzenie poprawności procesu tworzenia rezerwacji.
Warunki wstępne	Użytkownik jest zalogowany. Wyszukano dostępny lot.
Dane testowe	Dane pasażera oraz wybrany lot.
Kroki	1. Wyszukaj lot.2. Wybierz ofertę.3. Wprowadź dane pasażera.4. Wprowadź dane kontaktowe.5. Potwierdź rezerwację.
Oczekiwany rezultat	Rezerwacja zostaje zapisana w bazie danych, a użytkownik otrzymuje numer rezerwacji.
Wynik testu	Pozytywny

7.4 Testy jednostkowe

7.4.1 Test jednostkowy 1 – Tworzenie użytkownika

Cel: Sprawdzenie poprawności utworzenia nowego użytkownika.

Plik: tests/Unit/UserTest.php

```
<?php

namespace Tests\Unit;

use App\Models\User;
use App\Models\Role;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class UserTest extends TestCase
{
    use RefreshDatabase;

    public function test_user_can_be_created(): void
    {
        $role = Role::create([
            'name' => 'user'
        ]);

        $user = User::create([
```

```

        'role_id' => $role->id,
        'name' => 'Jan Kowalski',
        'email' => 'jan@test.pl',
        'password' => bcrypt('password123'),
    ]);

    $this->assertDatabaseHas('users', [
        'email' => 'jan@test.pl'
    ]);

    $this->assertEquals('Jan Kowalski', $user->name);
}
}

```

7.4.2 Wynik testu

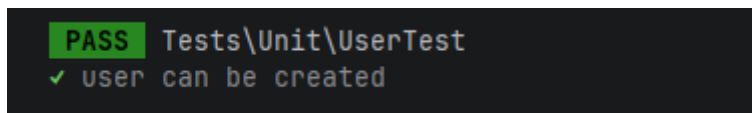


Figure 1: wynik UserTest

7.4.3 Test jednostkowy 2 – Status nowej rezerwacji

Cel: Sprawdzenie, czy nowa rezerwacja otrzymuje poprawny status.

Plik: tests/Unit/BookingTest.php

```

<?php

namespace Tests\Unit;

use App\Models\User;
use App\Models\Role;
use App\Models\Booking;
use App\Enums\BookingStatus;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class BookingTest extends TestCase
{
    use RefreshDatabase;

    public function test_booking_is_created_with_pending_status(): void
    {

```

```

        $role = Role::create([
            'name' => 'user'
        ]);

        $user = User::create([
            'role_id' => $role->id,
            'name' => 'Test User',
            'email' => 'test@test.pl',
            'password' => bcrypt('password'),
        ]);

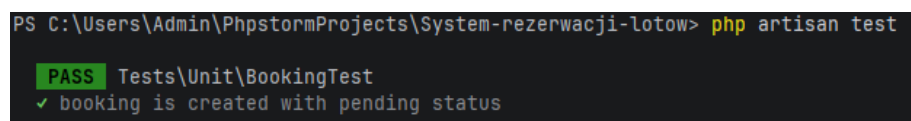
        $booking = Booking::create([
            'user_id' => $user->id,
            'booking_reference' => 'REF123',
            'status' => BookingStatus::PENDING,
            'passengers_count' => 1,
            'total_price' => 100,
            'currency' => 'USD',
        ]);

        $this->assertDatabaseHas('bookings', [
            'booking_reference' => 'REF123',
        ]);

        $this->assertEquals(BookingStatus::PENDING, $booking->status);
    }
}

```

7.4.4 Wynik testu



```

PS C:\Users\Admin\PhpstormProjects\System-rezerwacji-lotow> php artisan test

PASS Tests\Unit\BookingTest
✓ booking is created with pending status

```

Figure 2: wynik BookingTest

7.5 Testy integracyjne

Testy integracyjne zostały przygotowane w celu weryfikacji współpracy wielu komponentów systemu, takich jak modele, warstwa ORM (Eloquent) oraz baza danych.

Testy realizowane są z wykorzystaniem frameworka PHPUnit w środowisku Laravel oraz korzystają z mechanizmu `RefreshDatabase`, który zapewnia czyste środowisko testowe dla każdego uruchomienia.

7.5.1 Zakres testów integracyjnych

W ramach testów integracyjnych sprawdzono następujące scenariusze:

- tworzenie rezerwacji przez użytkownika systemu,
 - poprawność relacji pomiędzy użytkownikiem a rezerwacjami.
-

7.5.2 Test 1 – Tworzenie rezerwacji

Test sprawdza możliwość utworzenia rezerwacji powiązanej z użytkownikiem oraz poprawne zapisanie danych w bazie.

```
public function test_user_can_create_booking(): void
{
    $role = Role::create([
        'name' => 'user'
    ]);

    $user = User::create([
        'role_id' => $role->id,
        'name' => 'Integration User',
        'email' => 'integration@test.pl',
        'password' => bcrypt('password'),
    ]);

    $this->actingAs($user);

    $booking = Booking::create([
        'user_id' => $user->id,
        'booking_reference' => 'INT-001',
        'status' => 'pending',
        'passengers_count' => 2,
        'total_price' => 500,
        'currency' => 'USD',
    ]);

    // 5. assertions
    $this->assertDatabaseHas('bookings', [
        'booking_reference' => 'INT-001',
    ]);

    $this->assertEquals($user->id, $booking->user_id);
}
```

7.5.3 Test 2 – Relacja użytkownik → rezerwacje

Test weryfikuje poprawność relacji pomiędzy użytkownikiem a rezerwacjami.

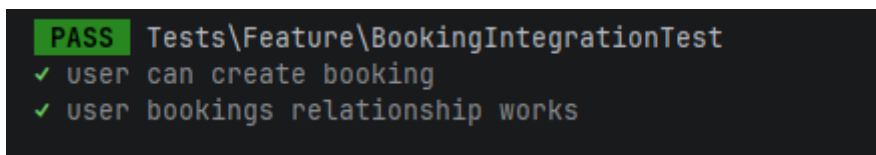
```
public function test_user_bookings_relationship(): void
{
    $role = Role::create(['name' => 'user']);

    $user = User::create([
        'role_id' => $role->id,
        'name' => 'Relation User',
        'email' => 'relation@test.pl',
        'password' => bcrypt('password'),
    ]);

    Booking::create([
        'user_id' => $user->id,
        'booking_reference' => 'REL-001',
        'status' => 'pending',
        'passengers_count' => 1,
        'total_price' => 100,
        'currency' => 'USD',
    ]);

    $this->assertCount(1, $user->fresh()->bookings);
}
```

7.5.4 Wynik testów integracyjnych



```
PASS Tests\Feature\BookingIntegrationTest
✓ user can create booking
✓ user bookings relationship works
```

Figure 3: wynik testów integracyjnych

#Dokumentacja techniczna

7.6 Instalacja i konfiguracja:

7.6.1 Continuous Integration i Continuous Deployment (CI/CD)

Projekt wykorzystuje GitHub Actions do automatycznego uruchamiania procesu Continuous Integration. Po każdym przesłaniu zmian do gałęzi **main** wykonywane są testy jednostkowe i integracyjne.

Platforma Railway została skonfigurowana z opcją **Wait for CI**, dzięki czemu

nowe wdrożenie następuje wyłącznie po pomyślnym zakończeniu wszystkich zadań GitHub Actions. W przypadku niepowodzenia testów proces wdrożenia zostaje automatycznie wstrzymany.

Takie rozwiązanie zapewnia, że na środowisko produkcyjne trafiają jedynie poprawnie zweryfikowane wersje aplikacji.

7.7 Implementacja zaplanowanych mechanizmów zapewniających bezpieczeństwo w praktyce

W projekcie zastosowano szereg mechanizmów bezpieczeństwa zgodnych z podejściami Secure by Design, Zero Trust oraz Privacy by Design.

7.7.1 1. Bezpieczeństwo danych w tranzycie i spoczynku

- Cała komunikacja z aplikacją odbywa się przez protokół HTTPS.
- Dane przesyłane pomiędzy klientem a serwerem są szyfrowane.
- Hasła użytkowników są przechowywane w formie haszowanej (bcrypt).

7.7.2 2. Autoryzacja i uwierzytelnianie

- System wykorzystuje mechanizm logowania oparty o Laravel Authentication.
- Dostęp do zasobów jest kontrolowany przez role użytkowników (np. administrator, użytkownik).
- Zastosowano zasadę najmniejszych uprawnień (Least Privilege).

7.7.3 3. Ochrona przed atakami

- Laravel automatycznie zabezpiecza aplikację przed SQL Injection poprzez ORM Eloquent.
- Zastosowano ochronę CSRF dla formularzy.
- Walidacja danych wejściowych odbywa się po stronie serwera.

7.7.4 4. Bezpieczeństwo infrastruktury

- Środowisko produkcyjne hostowane jest na platformie Railway.
- Dane konfiguracyjne (np. dostęp do bazy danych) przechowywane są w zmiennych środowiskowych, a nie w kodzie źródłowym.
- Dostęp do bazy danych jest ograniczony tylko do usług wewnętrznych aplikacji.

7.7.5 5. CI/CD i bezpieczeństwo wdrożeń

- Wdrożenie aplikacji następuje wyłącznie po pomyślnym przejściu testów w GitHub Actions.
- Każda zmiana w gałęzi main przechodzi proces weryfikacji (Continuous Integration).

- Mechanizm Railway “Wait for CI” blokuje wdrożenie w przypadku błędów testów.

7.7.6 6. Ochrona danych użytkownika (Privacy by Design)

- System minimalizuje zakres przetwarzanych danych osobowych.
- Dane użytkowników są wykorzystywane wyłącznie w zakresie niezbędnym do realizacji rezerwacji.
- Brak przechowywania danych kart płatniczych w systemie.

8 Dokumentacja użytkownika

8.1 Spis treści

1. Wprowadzenie
 2. Rejestracja i logowanie
 3. Wyszukiwanie lotów
 4. Proces rezerwacji lotu
 5. Zarządzanie rezerwacjami
 6. Płatności
 7. Generowanie biletu PDF
 8. Panel użytkownika
 9. Rozwiązywanie problemów
-

8.2 1. Wprowadzenie

System rezerwacji lotów umożliwia wyszukiwanie dostępnych połączeń lotniczych, tworzenie rezerwacji, zarządzanie pasażerami oraz generowanie biletów elektronicznych.

Aplikacja działa w przeglądarce internetowej i nie wymaga instalacji dodatkowego oprogramowania.

8.3 2. Rejestracja i logowanie

Użytkownik może założyć konto poprzez formularz rejestracyjny, podając: - imię i nazwisko, - adres e-mail, - hasło.

Po rejestracji możliwe jest logowanie do systemu przy użyciu adresu e-mail i hasła.

8.4 3. Wyszukiwanie lotów

System umożliwia wyszukiwanie lotów na podstawie: - lotniska wylotu, - lotniska przylotu, - daty podróży, - liczby pasażerów.

Wyniki można filtrować oraz sortować według ceny i czasu lotu.

9 4. Proces rezerwacji lotu

Proces rezerwacji rozpoczyna się po wybraniu konkretnego lotu z listy wyników wyszukiwania.

9.0.1 Krok 1 – wybór lotu

Użytkownik klika przycisk „Zarezerwuj”, aby przejść do formularza rezerwacji.

9.0.2 Krok 2 – dane pasażerów

Użytkownik wprowadza dane każdego pasażera: - imię i nazwisko, - data urodzenia, - obywatelstwo, - numer dokumentu.

System umożliwia dodanie wielu pasażerów w jednej rezerwacji.

9.0.3 Krok 3 – dane kontaktowe

Wymagane jest podanie: - adresu e-mail, - numeru telefonu.

9.0.4 Krok 4 – podsumowanie

System wyświetla: - szczegóły lotu, - listę pasażerów, - całkowity koszt rezerwacji.

9.0.5 Krok 5 – potwierdzenie

Użytkownik zatwierdza rezerwację i przechodzi do płatności.

10 5. Zarządzanie rezerwacjami

W panelu użytkownika dostępna jest lista wszystkich rezerwacji.

10.0.1 Dostępne operacje:

- podgląd szczegółów rezerwacji,
- anulowanie rezerwacji,
- sprawdzenie statusu płatności,

- pobranie biletu PDF.

10.0.2 Statusy rezerwacji:

- `pending` – oczekuje,
- `awaiting_payment` – oczekuje na płatność,
- `paid` – opłacona,
- `cancelled` – anulowana.

10.0.3 Anulowanie rezerwacji

Użytkownik może anulować rezerwację, jeśli nie została jeszcze wykorzystana. System może naliczyć opłatę zgodnie z warunkami taryfy.

10.1 6. Płatności

System obsługuje proces płatności dla rezerwacji. Po dokonaniu płatności status rezerwacji zmienia się na `paid`.

10.2 7. Generowanie biletu PDF

Po opłaceniu rezerwacji użytkownik może wygenerować bilet w formacie PDF zawierający: - dane pasażera, - numer rezerwacji, - szczegóły lotu.

10.3 8. Panel użytkownika

Panel użytkownika umożliwia: - edycję danych konta, - przegląd historii rezerwacji, - pobieranie dokumentów.

10.4 9. Rozwiązywanie problemów

W przypadku problemów: - sprawdź poprawność danych logowania, - upewnij się, że rezerwacja została opłacona, - odśwież stronę lub spróbuj ponownie później.